

Secure Image Messenger Using Steganography and Encryption



Mr. M Umair Khan

2217422

Mr. Seyam Ullah

2217415

BS Computer Science

Session 2022–2026

Supervisor: Prof. Mr. Karim Jaan

Department of Computer Science

Govt Degree College Gulabad

Affiliated with University of Malakand khyber, pakhtunkhwa, pakistan

Secure Image Messenger Using Steganography and Encryption

This Project Report is submitted in partial fulfillment of requirements for the degree of Bachelor in Computer Science to the Department of Computer Science, Govt Degree College Gulabad, affiliated with University of Malakand.

Submitted by:

Mr. M Umair Khan

Mr. Seyam Ullah

Certificate of Approval

It is certified that we have studied this project report submitted by Mr. M Umair Khan and Mr. Seyam Ullah in detail. We conclude that this report is of sufficient standard to warrant its acceptance by the Department of Computer Science, Govt Degree College Gulabad, affiliated with University of Malakand, for the award of the degree of BS in Computer Science.

Supervisor

Signature: _____

Name: Prof. Mr. Karim Jaan

Designation: _____

External Examiner

Signature: _____

Name: _____

Designation: _____

Institute: _____

Chairman, Department of Computer Science

Signature: _____

Name: _____

Dedication

This project is wholeheartedly dedicated to our beloved parents, who have been a source of inspiration and gave us strength, when we thought of giving up, who continually provide their moral, spiritual, emotional, and financial support.

To our brothers, relatives, friends, and classmates who shared their words of advice and encouragement to finish this project. And the last, we would like to dedicate our project to Allah Almighty who granted us good health and strength to accomplish this project. And we grateful to our supervisor Mr. Karim Jan Lecturer in Department of Computer Science, GDC Gulabad, University of Malakand. He guides us, encouraged us, for his help, advice and every correction throughout our project. We are also grateful to all our respected teachers.

Acknowledgements

All praise and gratitude belongs to Allah Almighty, whose guidance and blessings made this work possible. We are deeply grateful to our supervisor, Prof. Mr. Karim Jaan, for his consistent guidance, encouragement, and valuable feedback throughout this project. His expertise in computer science and his patience in addressing our questions shaped this work significantly.

We also extend our sincere thanks to the faculty and staff of the Department of Computer Science, Govt Degree College Gulabad, affiliated with University of Malakand, for providing a supportive academic environment. Special thanks to our families for their unwavering support and patience during the course of this project.

Finally, we acknowledge the open-source community behind Python, Django, and the Cryptography and Pillow libraries, whose tools formed the technical foundation of this system.

Table of Contents

List of Tables.....	v
List of Figures.....	vi
Glossary.....	vii
Abstract.....	viii
1. Introduction.....	1
1.1 Background.....	1
1.2 Motivation.....	2
1.3 Problem Statement.....	2
1.4 Objectives.....	3
1.5 Scope.....	3
1.6 Report Structure.....	4
2. Literature Review.....	5
2.1 Steganography: Origins and Techniques.....	5
2.2 Cryptography and Encryption Standards.....	5
2.3 Web-Based Secure Messaging Systems.....	6
2.4 Summary and Research Gap.....	6
3. System Design.....	7
3.1 System Architecture.....	7
3.2 Module Breakdown.....	7
3.3 Communication Flow.....	9
3.4 Security Design.....	9
3.5 Interface Design.....	10
4. Implementation.....	11
4.1 Development Environment.....	11
4.2 Encryption Implementation.....	11
4.3 Steganography Implementation.....	12
4.4 Web Application Implementation.....	12
4.5 Key Management.....	13
5. Testing and Results.....	14
5.1 Unit Testing.....	14
5.2 Integration Testing.....	14
5.3 Security Testing.....	15
5.4 Results Summary.....	15
6. Conclusion and Future Work.....	16
6.1 Conclusion.....	16
6.2 Future Work.....	16
References.....	18
Annexure A: System Screenshots.....	19
Copyrights Notice.....	20

List of Tables

Table 1.1 Technology Stack Summary..... 1

Table 5.1 Test Cases Summary..... 14

List of Figures

Fig. 3.1 System Architecture Diagram..... 7

Fig. 3.2 Sender Workflow Diagram..... 9

Fig. 3.3 Receiver Workflow Diagram..... 9

Glossary

AES (Advanced Encryption Standard): A symmetric block cipher adopted as an encryption standard, operating on 128-bit blocks with key sizes of 128, 192, or 256 bits.

Cryptography: The science of securing information by transforming it into an unreadable format using mathematical algorithms, readable only by authorised parties with the correct key.

Django: A high-level Python web framework that encourages rapid development and clean, pragmatic design, with built-in features for authentication, URL routing, and database management.

Fernet: A symmetric encryption scheme built on AES-128-CBC and HMAC-SHA256, provided by the Python Cryptography library, producing a self-contained encrypted token.

LSB (Least Significant Bit): The lowest-value bit in a binary representation. In image steganography, modifying the LSB of pixel colour values allows data to be hidden with negligible visual impact.

Steganography: The practice of concealing a message or file within another non-secret file or medium so that only the sender and intended recipient know of its existence.

Stego Image: An image that contains hidden data embedded using steganographic techniques. It appears visually identical to the original carrier image.

CSRF (Cross-Site Request Forgery): A web security vulnerability that allows attackers to trick users into performing unintended actions. Django protects against this using CSRF tokens.

PNG (Portable Network Graphics): A lossless image format used as the output format for stego images in this system to prevent compression from corrupting hidden data.

Abstract

Secure communication has become a critical need as digital threats grow more sophisticated. While encryption is the most widely used technique to protect sensitive data, it only secures the content of a message it does not hide the fact that secure communication is taking place. Attackers can still detect encrypted traffic, target it for interception, or subject it to traffic analysis. This creates a significant gap in how privacy is approached in current messaging systems.

This project, titled Secure Image Messenger Using Steganography and Encryption, addresses this gap by combining two complementary security techniques: cryptography and image-based steganography. The system first encrypts a text message using the Fernet encryption algorithm, which is built on AES-128 with HMAC authentication, producing a ciphertext that is unreadable without the correct key. This encrypted payload is then embedded inside a digital image using the Least Significant Bit (LSB) steganographic technique, effectively concealing the existence of any hidden communication within an ordinary image file.

The result is a two-layer security mechanism. Even if an attacker intercepts the image, they are unlikely to suspect it carries hidden data. And even if the steganographic layer is discovered, the encrypted content remains protected. The system is delivered as a web-based platform built using the Django framework, providing secure user authentication, a clean interface for message encoding and decoding, and safe file handling throughout. The platform is built using Python, the Cryptography library, Pillow for image processing, and SQLite for development storage.

Chapter 1

Introduction

The volume of digital communication exchanged daily has grown beyond what most people would have anticipated two decades ago. With this growth comes an equally rapid expansion in the threats targeting that communication. Surveillance, data interception, and unauthorised access are no longer theoretical concerns — they are documented, widespread, and increasingly sophisticated.

Encryption has long been the primary defence against such threats. When a message is encrypted, its content is transformed into an unreadable format that only an authorised recipient with the correct key can decode. However, encryption alone has a limitation that is often overlooked: it protects what is being said but not the fact that something is being said at all. An observer monitoring network traffic can see that two parties are exchanging encrypted messages, which itself can attract attention. Steganography addresses this limitation by concealing the very existence of the communication.

1.1 Background

Steganography is the practice of concealing information within ordinary, non-secret data so that the existence of a hidden message is not apparent. Digital image steganography is the most widely studied form, exploiting the redundancy present in the lower-order bits of pixel colour values. Modifying the Least Significant Bit (LSB) of each pixel channel causes no perceptible visual change but allows binary data to be encoded into an image. When combined with encryption, steganography creates a more complete security solution: the message is unreadable without the key, and its existence is concealed within an ordinary image [1][3].

1.2 Motivation

Most existing secure messaging tools either encrypt communication or apply steganography but rarely both within a single, user-friendly web-based system. Standalone steganography tools often embed plain text without any encryption, meaning the message is exposed once the hidden data is extracted. Encrypted messaging applications make no attempt to conceal the existence of secure communication. Neither approach alone is sufficient for scenarios requiring both confidentiality and covertness [4].

This project is motivated by the desire to fill that gap to build a working system that combines strong encryption, image steganography, and a proper web interface in a way that is technically sound and accessible to non-specialist users.

1.3 Problem Statement

Current messaging systems rely heavily on encryption to secure communication. While encryption protects message content, it does not conceal the act of communication itself. Encrypted traffic is identifiable and can be targeted, making users vulnerable to traffic analysis and suspicion-based surveillance [1].

Existing steganographic tools are often standalone applications that lack integrated encryption and do not provide a centralised web-based platform. Most embed plain text messages, leaving content exposed if the hidden data is ever detected [4]. There is therefore a clear need for an integrated solution that both encrypts message content and conceals its existence within a digital medium, delivered through a secure and accessible web platform.

1.4 Objectives

The primary aim of this project is to design and develop a secure web-based messaging system combining encryption and image steganography for covert and confidential communication. The specific objectives are:

- i. To implement Fernet encryption to protect plaintext messages before embedding into images.
- ii. To develop an LSB-based image steganography module using Pillow for hiding encrypted data within digital images.
- iii. To build a message extraction and decryption workflow allowing the receiver to recover the original message using a shared secret key.
- iv. To design a Django-based web application with user authentication, secure session management, and safe file handling.
- v. To implement a key management system that generates encryption keys securely.
- vi. To build a security audit log that records all system events for monitoring purposes.
- vii. To test the system for functional correctness, security, and usability.

1.5 Scope

This project covers the full design and development of a web-based secure image messenger. It includes the encryption module, the steganography module, the extraction and decryption pipeline, and the supporting web application with user authentication, dashboard, operation history, and key management. The system is developed and tested in a local environment using SQLite and is designed to be deployable to cloud platforms using PostgreSQL. The scope does not extend to mobile application development or real-time messaging features, which are identified as potential areas for future work.

1.6 Report Structure

Chapter 2 reviews related literature on steganography, encryption, and web-based secure systems. Chapter 3 describes the system design including architecture, modules, and workflow. Chapter 4 covers the implementation of each component. Chapter 5 presents the testing strategy and results. Chapter 6 concludes the report and discusses future directions.

Table 1.1 summarises the technology stack used in this project.

Component	Technology
Backend Framework	Python / Django 5.2
Encryption	Fernet (AES-128-CBC + HMAC-SHA256)
Image Processing	Pillow 10
Frontend	HTML5, CSS3, Bootstrap 5
Database (Dev)	SQLite
Database (Prod)	PostgreSQL
Deployment	PythonAnywhere / Render

Table 1.1 Technology Stack Summary

Chapter 2

Literature Review

Research in steganography, cryptography, and secure communication has produced a rich body of work over several decades. This chapter reviews the most relevant contributions informing the design of this project.

2.1 Steganography: Origins and Techniques

Johnson and Jajodia [1] provided one of the earliest comprehensive surveys of digital image steganography, establishing LSB substitution as a practical and effective technique. They noted that digital images contain a large amount of redundant data, particularly in the lower-order bits of pixel colour values, making them a natural carrier for hidden information with no perceptible quality loss.

Cheddad et al. [3] extended this foundation with a detailed survey comparing spatial domain techniques such as LSB with transform domain approaches including DCT and DWT-based methods. While transform domain techniques offer greater robustness against detection, spatial domain LSB methods remain the most efficient and practical for applications where imperceptibility is the primary concern and the threat model does not include active steganalysis. For this project, LSB substitution represents the most appropriate choice given the balance between simplicity, performance, and imperceptibility.

2.2 Cryptography and Encryption Standards

Schneier [4] offers a comprehensive reference on cryptographic protocols and algorithms, establishing AES as the standard for symmetric encryption with proven strength against brute force and known cryptanalytic attacks. The Fernet encryption scheme used in this project is built on AES-128-CBC with HMAC-SHA256 authentication, providing both confidentiality and integrity verification properties directly relevant to secure message transmission.

The combination of encryption and steganography has been explored across multiple studies as a means of achieving layered security. The general finding is that neither technique alone is sufficient for high-assurance covert communication. Encryption without steganography reveals the existence of secure communication. Steganography

without encryption exposes message content once the carrier is analysed. Their combination addresses both weaknesses simultaneously [1][3].

2.3 Web-Based Secure Messaging Systems

The Django framework [2] provides a mature, production-ready platform with built-in security features relevant to this project, including CSRF protection, secure session management, PBKDF2 password hashing, and a robust authentication system. These features reduce the implementation burden for security-critical web applications significantly.

Existing web-based steganography tools generally fall into two categories: those that provide steganographic embedding without encryption, or those that provide encrypted messaging without any steganographic layer. Tools such as online LSB encoders embed user-provided text but store and transmit messages in plain form. This project addresses the absence of an integrated, web-based platform combining both techniques within a single authenticated system.

2.4 Summary and Research Gap

The literature confirms that both encryption and steganography are individually well-understood techniques. However, few systems successfully integrate both within a user-accessible, web-based platform. Most existing implementations are either academic prototypes or standalone desktop applications with limited accessibility. The gap this project fills is the absence of a functional, web-delivered messaging system combining AES-based encryption, LSB steganography, secure authentication, and key management in a single coherent application.

Chapter 3

System Design

This chapter describes the overall architecture of the system, the design of each module, and the communication flow between components. The design prioritises modularity, security, and usability.

3.1 System Architecture

The system follows the Model-View-Template (MVT) architecture prescribed by Django. The model layer handles database interaction through Django's ORM. The view layer contains business logic and invokes supporting modules. The template layer manages the user interface. Encryption and steganography functions are implemented as standalone Python modules, keeping core processing logic independent of the web layer.

At a high level, a user interacts with the system through a browser. Requests are handled by Django's URL dispatcher, which routes them to the appropriate view function. The view invokes the relevant modules and returns a rendered HTML response. Images are stored in a designated media directory. All database operations are performed through Django's ORM against an SQLite database in development.

Fig. 3.1 below illustrates the system architecture.

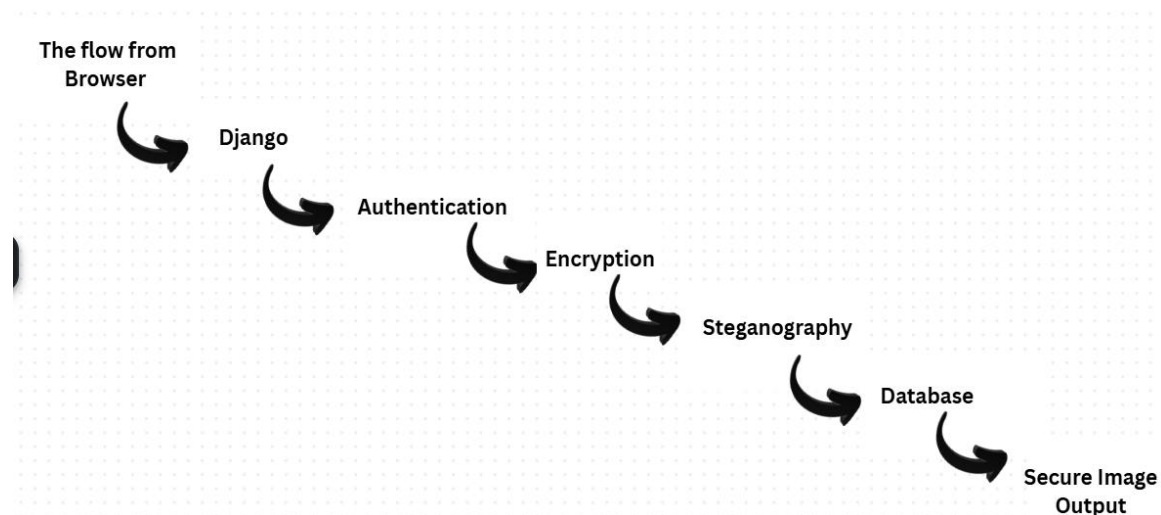


Fig. 3.1 System Architecture Diagram

3.2 Module Breakdown

The system is divided into the following core modules:

3.2.1 Encryption Module (encryption.py)

The encryption module provides functions for key generation, Fernet-based encryption, and decryption. It also includes an AES-GCM implementation for advanced use cases. The module exposes clean function interfaces — `encrypt`, `decrypt`, `generate key` — called by the view layer without exposing implementation details. The Fernet scheme produces a self-contained encrypted token that includes the ciphertext, IV, and HMAC, making it resistant to tampering.

3.2.2 Steganography Module (steganography.py)

The steganography module implements LSB embedding and extraction using the Pillow image processing library. The `embed` function accepts the raw bytes of a carrier image and the encrypted payload string, modifies the least significant bit of each RGB channel sequentially, and returns the stego image as PNG bytes. The `extract` function reads the LSBs and reconstructs the payload up to a predefined delimiter. A capacity checking function calculates the maximum payload size a given image can carry, expressed as $(\text{width} \times \text{height} \times 3) \div 8$ bytes.

3.2.3 Web Application (Django)

The web application handles user registration, login, logout, session management, and page rendering. Views are function-based and map directly to URL patterns. Each authenticated view verifies the session before processing. File uploads are validated for type and size before any processing occurs. All forms include CSRF tokens, and the application is configured with Django's security middleware.

3.2.4 Database Models

Three main models are defined. `UserProfile` extends Django's built-in `User` model with fields for operator identity and security preferences. `StegOperation` records each encoding or decoding action, including image references, encryption method, and operation status. `SecurityLog` maintains an audit trail of all security-relevant events including logins, key generation, and failed decryption attempts.

3.3 Communication Flow

The sender's workflow proceeds as follows. The sender navigates to the `Encrypt` page and types a plaintext message. The view generates a Fernet key, encrypts the message, and returns the ciphertext and key to the sender. The sender copies the key for secure out-of-band sharing. On the `Embed` page, the sender uploads a carrier image. The view calls the

steganography module, embeds the ciphertext, saves the stego image, and offers it for download. The stego image is shared through any standard channel.

The receiver's workflow proceeds as follows. The receiver uploads the stego image on the Extract page. The view reads the image bytes, calls the extraction function, and returns the ciphertext. The receiver navigates to the Decrypt page, pastes the ciphertext, enters the secret key, and the view decrypts and displays the original message.

Fig. 3.2 and Fig. 3.3 illustrate the sender and receiver workflows respectively.

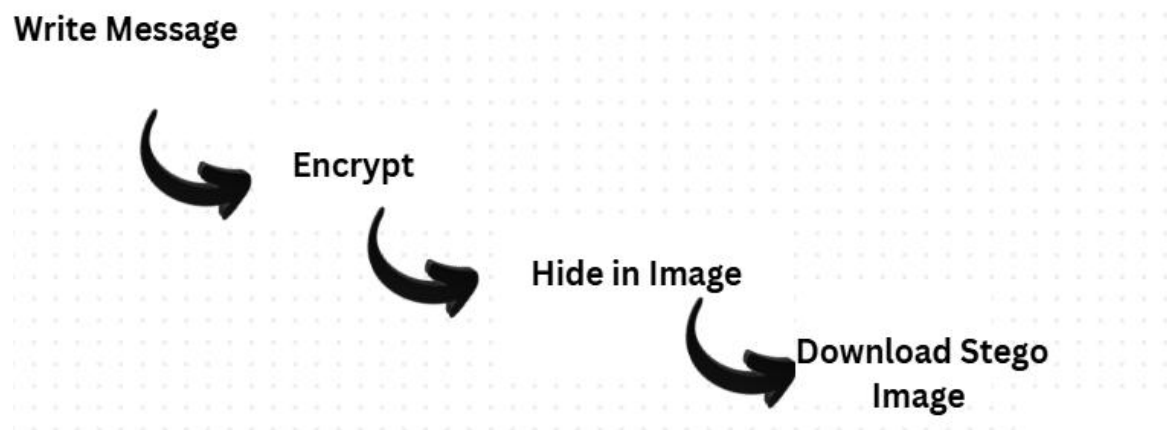


Fig. 3.2 Sender Workflow Diagram

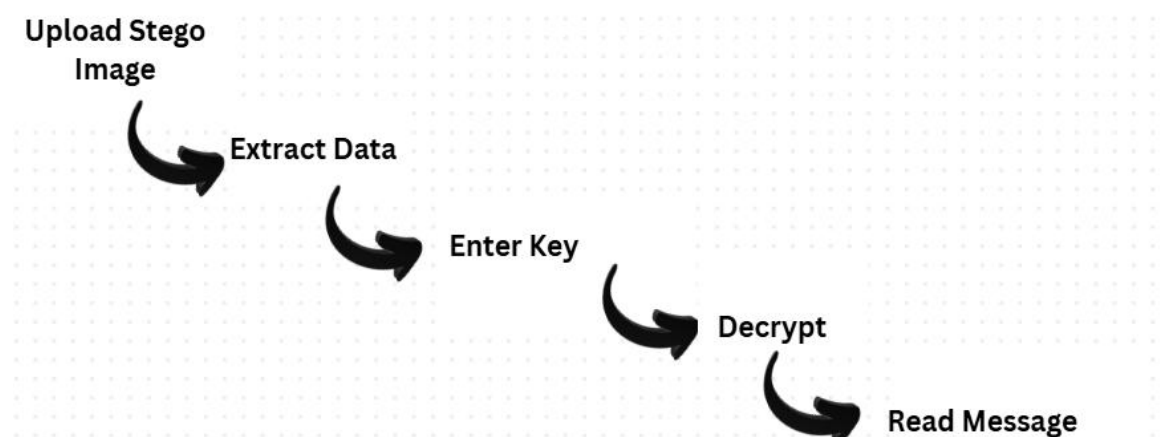


Fig. 3.3 Receiver Workflow Diagram

3.4 Security Design

Security is built into the system at multiple levels. Passwords are hashed by Django's authentication system using PBKDF2-SHA256. All forms include CSRF protection tokens. File uploads are validated for extension, MIME type, and size before processing. Secret keys are never stored on the server displayed once at generation and the user's responsibility to preserve. Sessions use HTTP-only cookies with configurable timeout. All security events are recorded to the SecurityLog model.

3.5 Interface Design

The user interface uses a dark cybersecurity theme with Bootstrap 5 for responsive layout. A fixed sidebar provides navigation, with a main content area for page-specific content. Dashboard widgets display operation statistics and security status. All forms use consistently styled inputs. The interface functions on both desktop and mobile browsers.

Chapter 4

Implementation

This chapter describes the implementation of each system component, covering the key technical decisions, tools, and patterns used throughout development.

4.1 Development Environment

The system was developed on Windows 10 using Python 3.11 and Django 5.2. VS Code was the primary development environment. Core libraries include the Python Cryptography library [5] for encryption, Pillow 10 [6] for image processing, and WhiteNoise for static file serving. SQLite was used as the development database, with the system designed to be compatible with PostgreSQL for production deployment.

4.2 Encryption Implementation

The Fernet class from the Python Cryptography library was selected as the primary encryption mechanism. Fernet uses AES-128-CBC for confidentiality and HMAC-SHA256 for integrity verification, producing a self-contained token safe for storage and transmission. Key generation uses `Fernet.generate_key()`, which produces a cryptographically secure 32-byte URL-safe base64 encoded key.

The encryption function accepts a plaintext string and key string, instantiates a Fernet object, and returns the encrypted token as a UTF-8 string. The decryption function performs the reverse, catching the `InvalidToken` exception raised when the key is incorrect or the token has been tampered with, and raising a descriptive `ValueError` for the view layer to handle gracefully. An optional AES-256-GCM implementation is also provided for applications requiring a larger key size.

4.3 Steganography Implementation

The LSB steganography module is implemented entirely using Pillow, without dependency on OpenCV. The `embed` function opens the carrier image, converts it to RGB mode, and iterates through each pixel. For each pixel, it modifies the LSB of the red, green, and blue channels sequentially to encode the payload bits. A fixed delimiter string is appended to the payload to mark its end. The modified pixel data is saved as a PNG file to prevent lossy compression from corrupting the hidden data.

The extract function reads the LSBs from each pixel channel in the same sequential order, reconstructs bytes from groups of eight bits, and decodes them as UTF-8 characters. The function terminates as soon as the delimiter is detected, avoiding the need to process the entire image. A capacity checking function calculates the maximum payload as $(\text{width} \times \text{height} \times 3) \div 8$ bytes, allowing the view to warn users if the message exceeds the image capacity.

4.4 Web Application Implementation

The web application follows Django conventions. The project package contains settings, URL configuration, and WSGI entry point. The messenger application contains models, views, forms, URLs, and supporting utility modules. Templates are stored in a top-level templates directory and static files are served through WhiteNoise.

Environment-specific settings including the Django secret key, debug flag, and database URL are stored in a .env file loaded at startup using python-dotenv, keeping sensitive configuration out of version control. All views use Django's login_required decorator to redirect unauthenticated users to the login page. File uploads are validated using a custom function checking extension, MIME type, and file size before any processing.

4.5 Key Management

The settings page allows authenticated users to generate new Fernet keys on demand. Generated keys are displayed once and never stored in the database. Users are warned explicitly to copy and preserve the key before proceeding, as there is no recovery mechanism. Key generation events are recorded in the SecurityLog model. A client-side AJAX endpoint validates the format of a key as the user types in the decrypt form, providing immediate feedback before form submission.

Chapter 5

Testing and Results

Testing was conducted across each module and across the integrated system to verify functional correctness, security behaviour, and usability. This chapter summarises the testing approach and results.

5.1 Unit Testing

The encrypt and decrypt functions were tested with input lengths ranging from single characters to messages several thousand characters long. All encrypted outputs were verified to be unreadable ciphertext. All decryption operations with the correct key returned the exact original plaintext. Decryption with an incorrect key consistently raised a `ValueError`, confirming that the HMAC integrity check was functioning correctly.

The embed and extract functions were tested with images of varying dimensions and payloads of varying sizes. For each test case, the extracted payload matched the original input exactly. Visual inspection of stego images confirmed no perceptible difference from the source images. Capacity checking was tested with oversized payloads to confirm that the appropriate error was raised before embedding was attempted.

5.2 Integration Testing

The full encode and decode workflow was tested end-to-end. A plaintext message was entered on the Encrypt page, producing a ciphertext and key. The ciphertext was embedded into a PNG image on the Embed page, and the stego image was downloaded. The stego image was uploaded on the Extract page, producing the ciphertext. The ciphertext and key were entered on the Decrypt page, recovering the original plaintext. The recovered message matched the original in every test case.

Cross-format testing confirmed that JPEG source images are correctly converted to PNG at the embedding stage, preventing JPEG compression from corrupting the hidden data. Images downloaded and re-uploaded without modification consistently yielded correct extraction results.

Table 5.1 summarises the test cases and results.

Test Case	Description	Expected	Result
Encryption	Encrypt plain text with valid key	Ciphertext returned	Pass
Decryption	Decrypt with correct key	Original text returned	Pass
Wrong Key	Decrypt with incorrect key	ValueError raised	Pass
LSB Embed	Embed payload in PNG image	Stego image generated	Pass
LSB Extract	Extract from stego image	Exact payload returned	Pass
Capacity Check	Oversized payload	Error before embedding	Pass
CSRF	Submit form without token	403 response	Pass
File Validation	Upload non-image file	Rejected with error	Pass
Auth Guard	Access page without login	Redirect to login	Pass

Table 5.1 Test Cases Summary

5.3 Security Testing

CSRF protection was verified by submitting forms without valid tokens, which were correctly rejected with 403 responses. File upload validation was tested by uploading non-image files and oversized images, both rejected with appropriate error messages. Session security was verified by attempting to access authenticated pages without a valid session, resulting in consistent redirection to the login page. Incorrect decryption keys were handled gracefully with informative error messages rather than unhandled exceptions.

5.4 Results Summary

All core functions performed correctly across test cases. The system successfully encrypts, embeds, extracts, and decrypts messages without data loss. All security controls behaved as designed. No critical defects were identified during testing. The interface was reviewed for usability and found to be clear and intuitive for the primary workflows.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project has successfully delivered a working web-based secure image messenger combining Fernet encryption and LSB image steganography within a Django application. The system provides a complete workflow for both senders and receivers: encrypting messages, embedding them in carrier images, extracting hidden payloads, and decrypting the recovered data. All operations are performed through a secure, authenticated web interface with CSRF protection, input validation, safe file handling, and a full security audit log.

The dual-layer security approach addresses a real limitation of encryption-only messaging systems. By concealing both the content and the existence of a message, the system provides a stronger privacy guarantee than either technique could achieve alone. The implementation is modular and well-documented, making it a solid foundation for further development.

The project meets all objectives set out at the start. Encryption is implemented and functional. The steganography module embeds and extracts data accurately without visible image degradation. The web application is secure and user-friendly. Key management is handled appropriately. The system has been tested and performs reliably across all test cases.

6.2 Future Work

Several directions for future development were identified during this project:

- **Audio and Video Steganography:** Extending the steganography module to support audio files (WAV) and video files would significantly expand the system's capabilities and resistance to detection.
- **AES-256 as Default:** Migrating to AES-256-GCM as the default encryption algorithm would increase the key size and provide authenticated encryption with associated data (AEAD) properties appropriate for high-security applications.

- **Real-Time Messaging:** Integrating Django Channels to support WebSocket-based real-time messaging would allow the system to function as a live communication platform rather than a file-based exchange system.
- **Steganalysis Resistance:** Implementing adaptive steganographic algorithms such as HUGO or WOW would make the system significantly more resistant to statistical steganalysis tools.
- **Mobile Application:** Developing a mobile client for Android or iOS would extend accessibility and allow the system to be used in a wider range of real-world scenarios.
- **Two-Factor Authentication:** Adding time-based one-time password (TOTP) two-factor authentication would strengthen user account security beyond the current password-based login.

These extensions would transform the current proof-of-concept into a more complete, production-ready secure communication platform. The modular architecture of the current implementation is designed with these extensions in mind.

References

- [1] N. F. Johnson and S. Jajodia, "Exploring Steganography: Seeing the Unseen," IEEE Computer, vol. 31, no. 2, pp. 26–34, 1998.
- [2] Django Software Foundation, "The Web Framework for Perfectionists with Deadlines," Django Documentation, 2025. [Online]. Available: <https://www.djangoproject.com/>
- [3] A. Cheddad, J. Condell, K. Curran, and P. McKevitt, "Digital Image Steganography: Survey and Analysis of Current Methods," Signal Processing, vol. 90, no. 3, pp. 727–752, 2010.
- [4] B. Schneier, Applied Cryptography: Protocols, Algorithms, and Source Code in C, 20th Anniversary Ed. Wiley, 2015.

[5] Python Cryptography Developers, "Cryptography Documentation," 2024. [Online]. Available: <https://cryptography.io/en/latest/>

[6] Pillow Contributors, "Pillow (PIL Fork) Documentation," 2024. [Online]. Available: <https://pillow.readthedocs.io/>

Annexure A: System Screenshots

This annexure contains screenshots of the main pages of the SecureStego Messenger web application, demonstrating the user interface and core functionality.

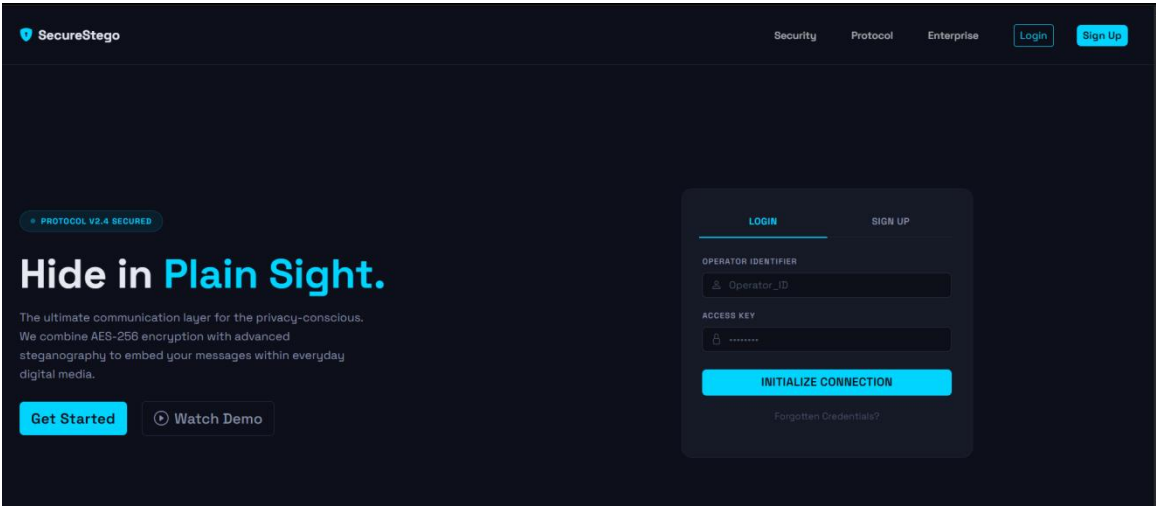


Fig. A.1 Home Page

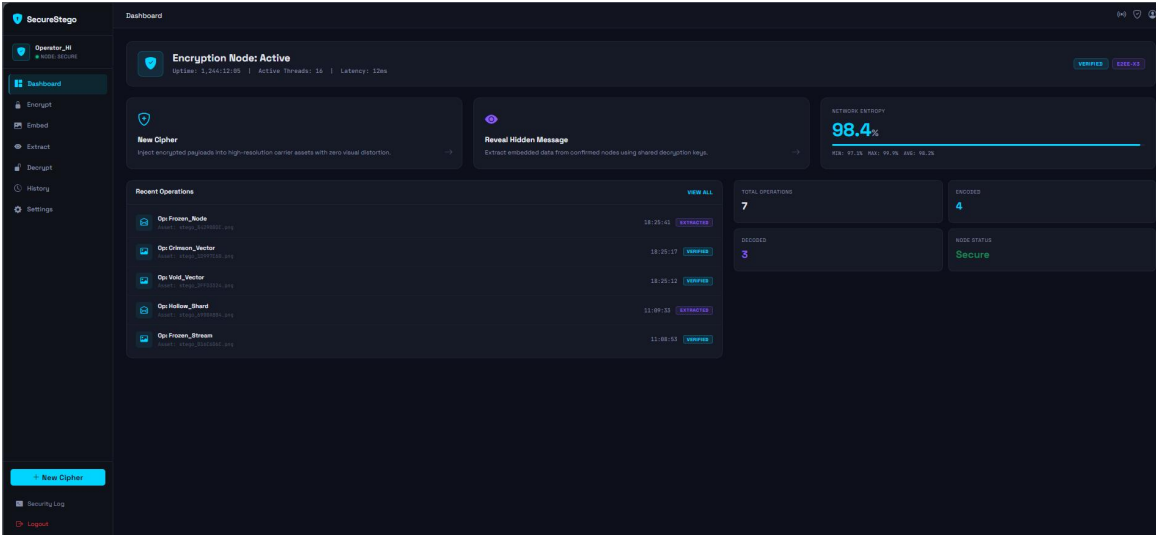


Fig. A.2 User Dashboard

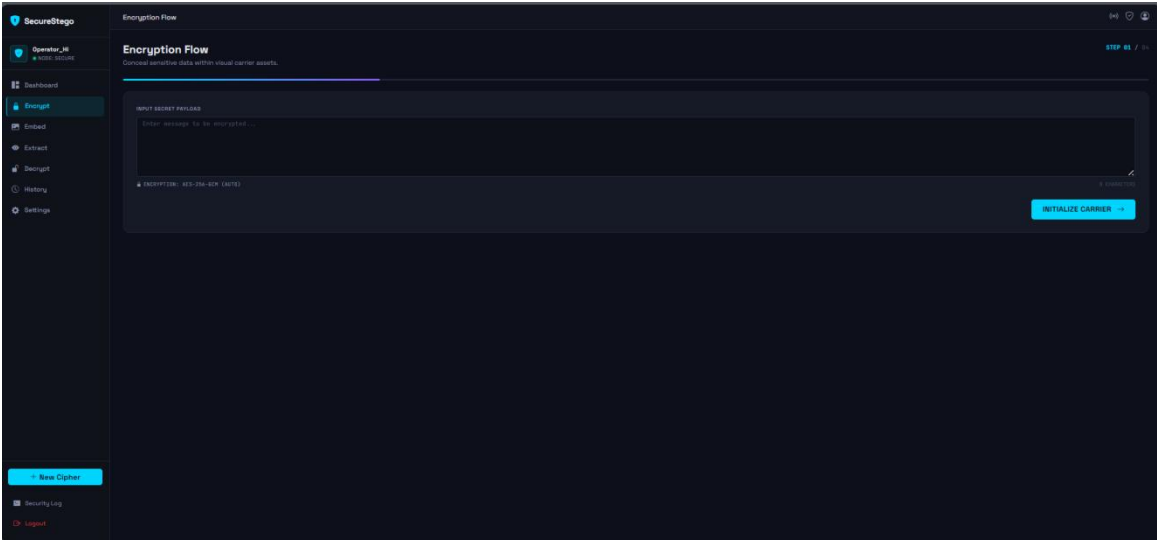


Fig. A.3 Encrypt Message Page

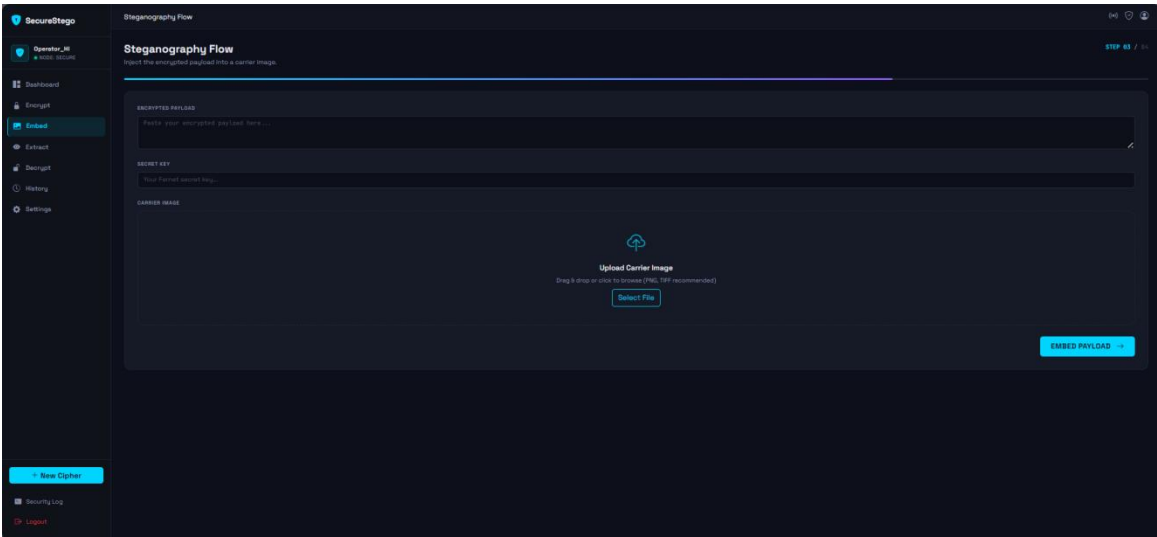


Fig. A.4 Embed Message in Image Page

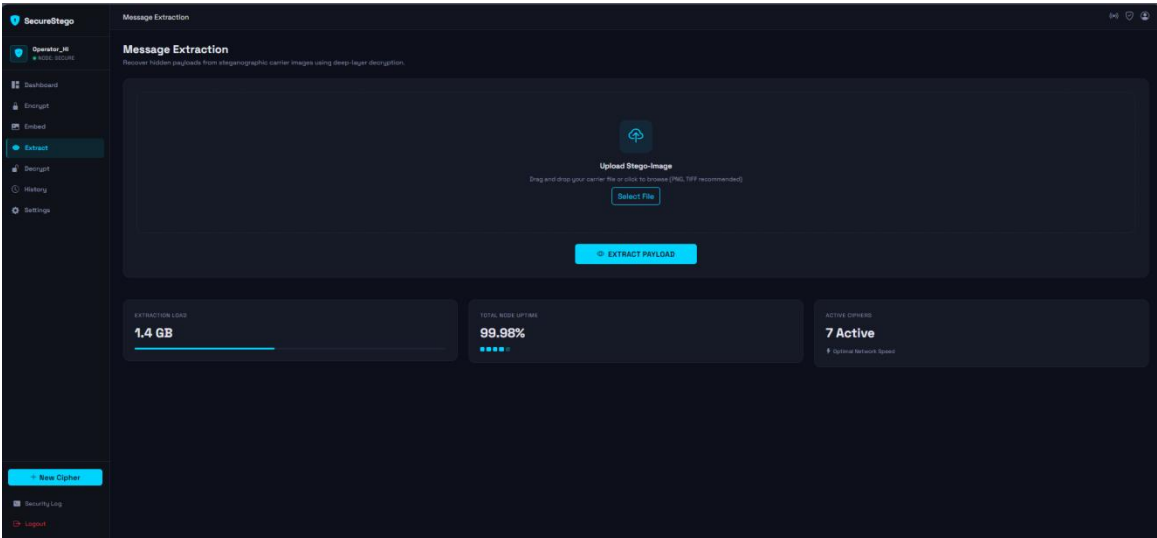


Fig. A.5 Extract Hidden Message Page

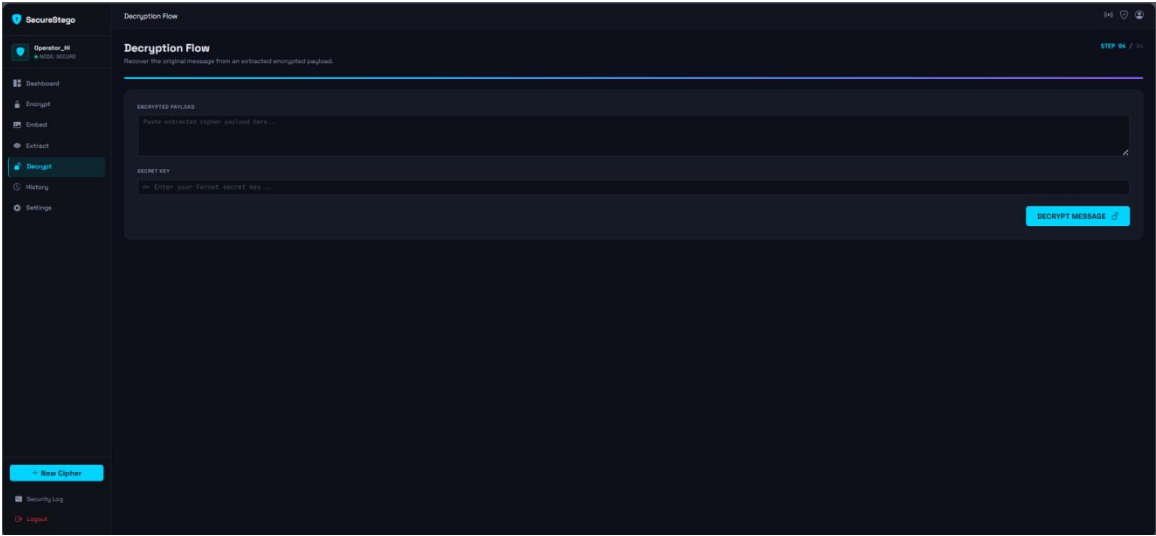


Fig. A.6 Decrypt Message Page

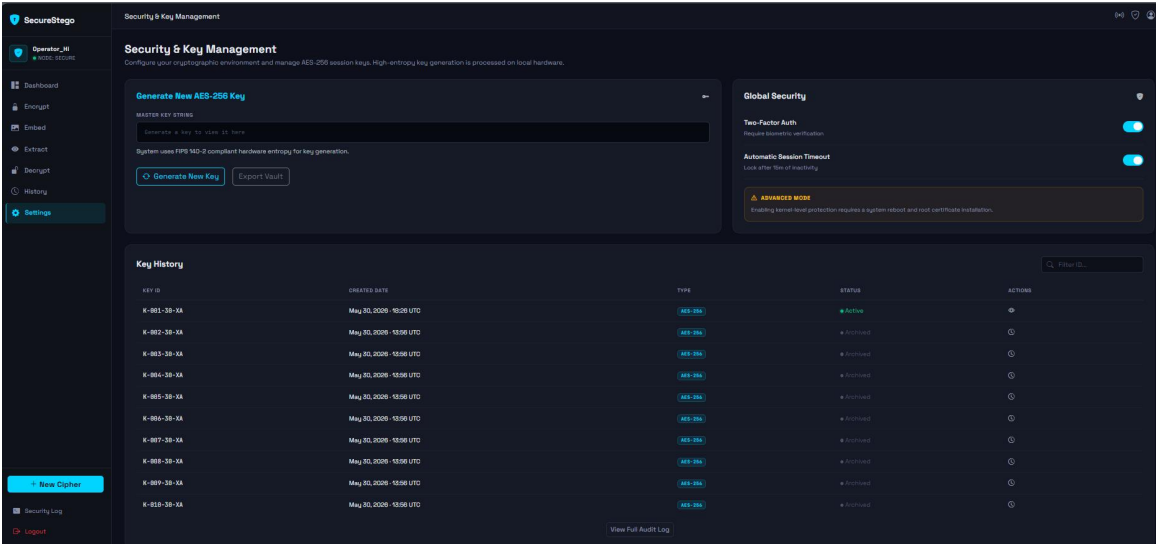


Fig. A.7 Security and Key Management Page

Copyrights Notice

This dissertation under the title Secure Image Messenger Using Steganography and Encryption is the intellectual property of Mr. M Umair Khan and Mr. Seyam Ullah.

No part of this document may be reproduced, stored in any retrieval system or used otherwise without the prior permission of the authors. We hereby allow the University of Malakand to make copies of this dissertation for academic purposes only.

Any individual or organisation may make copies of this document for non-commercial purposes with prior permission of the authors.

Mr. M Umair Khan

BS Computer Science

Mr. Seyam Ullah

BS Computer Science